Advances in Cognitive Systems 13 (2026) Submitted 8/2026; published 10/2026

SMTB: Fast Structure-Mapping with Tight Bounds

Daniel Weitekamp WEITEKAMP@GATECH.EDU
Christopher MacLellan CMACLELL@GATECH.EDU
School of Interactive Computing, Georgia Tech, Atlanta, GA 30332 USA

Abstract

Structure-mapping forms analogies by aligning systems of relationally connected elements based on shared structure instead of surface features. We introduce a new structure-mapping algorithm: Structure-Mapping with Tight Bounds (SMTB) that is 5–15x faster than the structure-mapping engine (SME) and about 50% better at finding mappings in large nested domains. SMTB is part of the broader Cognitive Rule Engine (CRE) project, a flexible multi-language-compatible framework with an accessible Python interface to state-of-the-art C++ implementations of core algorithms commonly used in cognitive systems such as pattern matching, planning, and structure-mapping. CRE and SMTB are designed to work with a wide range of representation choices. Unlike SME, which biases higher-order correspondences in tree-like predicate logic, SMTB maximizes relational connectivity without privileging higher-order relations. This allows SMTB to work just as well over arbitrary relational graphs as it does in tree-like domains of nested predicate logic. We discuss situations where privileging "higher-orderness" in structure-mapping can cause issues, and illustrate how SMTB avoids failure modes that SME would encounter in these situations. We also provide an evaluation comparing SMTB to SME v4 over 5845 domain pairs from the SME corpus.

1. Introduction

Central to analogical reasoning is the ability to recognize relational similarities between things (e.g., factual information, objects in natural scenes, or mental abstractions) based on their shared relational structure. This is an essential mechanism in human cognition that mediates sense-making and learning (Gentner, 2010) and allows us to reason about new, uncertain situations by mapping them onto known ones. *structure-mapping* (Gentner, 1983) is a computational theory of the formation of structural analogies, which characterizes analogies as self-consistent one-to-one mappings between *base* and *target* domains of relationally encoded entities. In cognitive systems, structure-mapping has many uses (Gentner & Forbus, 2011), including drawing analogies between experiences, aligning knowledge structures to combine or relax them into higher-order generalizations, and doing partial matching to assess the degree of applicability of prior knowledge in new situations. In this work, we present structure-mapping with Tight Bounds (SMTB) and demonstrate areas where it improves upon the structure-mapping engine (SME) (Falkenhainer et al., 1989)—a system that has served as the structure-mapping implementation for 40 years of cognitive systems, including several ongoing projects (Gentner & Forbus, 2025). SME has been used in cognitive systems such as SAGE (Kuehne et al., 2000; McLure et al., 2015) and CogSketch (Forbus et al., 2011), in the Companion

© 2026 Cognitive Systems Foundation. All rights reserved.

Base: water flow	**Target: heat flow**
Greater$_1$(Pressure(beaker), Pressure(vial))	Greater(Temp(coffee), Temp(icecube))
Greater$_2$(Diameter(beaker), Diameter(vial))	
Flow(beaker, vial, water, pipe)	Flow(coffee, icecube, heat, bar)
Cause(Greater$_1$(...), Flow(...))	
Liquid(water)	Liquid(coffee)
Flat-Top(water)	

Table 1: Water-flow / heat-flow analogy (Falkenhainer et al., 1989). Colors mark the correspondences of the intended mapping between base entities: beaker→coffee, vial→icecube, water→heat, pipe→bar, and expressions: Greater$_1$→Greater, Flow→Flow, Pressure→Temp. Greater$_1$ and Greater$_2$ are distinct instances of Greater, over Pressure, and Diameter.

cognitive architecture (Forbus & Hinrichs, 2017), and in studies of analogical reasoning in solving AP physics problems (Klenk & Forbus, 2009).

We began the development of SMTB to serve as a fast, Python-accessible structure-mapping implementation for our students working on our Python-based cognitive systems. During development, we found that SME's approach imposed constraints and search biases that explicitly precluded structure-mapping solutions that were desirable in our use cases. These issues, which we expand on in Section 3, motivated SMTB's approach which relaxes some of SME's constraints.

1.1 Water Flow → Heat Flow Example

Falkenhainer et al. (1989) provide the conical structure-mapping example, which draws an analogy between water flow driven by differences in pressure and heat flow driven by differences in temperature (shown in Table 1). Base entity identifiers (e.g., beaker, vial, water, pipe) are connected to one another through hierarchies of relational expressions encoded as predicates (e.g., Greater, Pressure, Diameter, Flow). The base encodes a situation where "water flows from a beaker to a vial through a pipe, because the pressure in the beaker is greater". In the target: "heat flows from coffee to an ice cube through a bar". A successful mapping between them enables inference of a missing idea in the target: the flow occurs "because the temperature in the coffee is greater than the ice cube". Table 1 is aligned and colored to illustrate the ideal analogical mapping for this case. The ideal mapping skips shallow attribute correspondences, such as water and coffee both being Liquid, and instead identifies stronger structural alignment in higher-order relations. A naive commitment to water→coffee (because they are both liquid) would have prevented the higher-order Greater$_1$→Greater, and Flow→Flow mappings. In general, structure-mapping systems such as SME can optionally yield multiple mappings that can differ in their assigned alignment score and ground-truth quality. For instance, Greater$_2$→Greater along with Diameter→Temp is another logically consistent mapping—albiet one that blocks a candidate inference of a cause in the target, and thus may have a lower score.

1.2 Our Motivations

Computational Models of Learning: SMTB was developed as a fast, flexible structure-mapping system for computational models of learning (Weitekamp & Koedinger, 2023) such as the Apprentice Learner (Maclellan et al., 2016) and AI2T (Weitekamp et al., 2024), which simulate human learning from intelligent tutoring system instruction. Unlike statistical simulations of student performance change (Cen et al., 2006) and recent attempts to replicate student behavior with LLMs (Käser & Alexandron, 2024), AI2T and the Apprentice Learner master procedural academic tasks through rapid bottom-up induction from the limited instructional experiences available in automated tutoring systems. Beginning with limited prior knowledge, they learn domain expertise, often with 12 or fewer practice problems. Structure-mapping plays a critical role in enabling this data-efficient, human-like learning, which is many orders of magnitude faster than neural approaches like reinforcement learning and requires no external pretrained knowledge sources such as LLMs (Weitekamp et al., 2025a). Running these teachable simulated learners at scale (Weitekamp et al., 2025b), and in real-time for applications where they are taught interactively from human instruction (Weitekamp et al., 2020, 2024), requires efficient and flexible subsystems for pattern matching and structure-mapping that work robustly over diverse environment representations, and which do not lag or break when applying the often complex or underconstrained knowledge structures induced by these simulated learners.

The Cognitive Rule Engine (CRE)[1]**,** the toolset in which SMTB is implemented, was initially developed to meet the challenging computational needs of these simulated learners. Prior publications have reported on CRE's incremental pattern-matching algorithm CORGI (Weitekamp, 2025), which has quadratic first-match complexity guarantees that eliminate the exponential blowup of RETE-based and naive unification-based matching and beat well-optimized implementations such as SOAR (Laird, 2019) by orders of magnitude across all scales. Structure-mapping with Tight Bounds (SMTB) extends CRE with structure-mapping. We show that, like CORGI, SMTB is fast and flexible—it is 5–15x faster than SME on different problem sizes and works in domains with cycles instead of strictly tree-like nested predicate logic.

CRE has grown into a broader project that aims to serve as a public open-source toolset for cognitive systems researchers. The aim is to accelerate cognitive system development, similar to how PyTorch and TensorFlow have accelerated deep learning research. CRE is implemented as a C++ framework that can be called from wrapper languages such as Python. Instead of specifying prior knowledge and domain content solely in a domain-specific language (e.g., PDDL or .soar files), CRE is designed with dynamic, first-class usage in mind: all data and knowledge structures, such as matching patterns and rules, can be flexibly specified directly in wrapper languages like Python. All objects created this way are fast C++ data structures; however, they are natively inspectable and editable within the user-facing wrapper language. This setup supports rapid development of learning mechanisms that treat knowledge as a flexibly inducible and modifiable material, rather than something that is only prespecified or recombined, as in speed-up learning approaches common in ACT-R and SOAR (Anderson et al., 1997; Laird, 2019). Our hope is that this flexible framework-oriented design, combined with fast implementations of common algorithms like pattern matching

1. https://github.com/DannyWeitekamp/Cognitive-Rule-Engine/

and structure matching, will help facilitate rapid iteration in cognitive systems research and help scale beyond the scattered toy implementations and monolithic architecture-locked implementations (e.g., ACT-R, SOAR) that have dominated cognitive systems research for so long.

2. The structure-mapping Engine (SME)

2.1 structure-mapping Theory

SME is built on Structure-Mapping Theory (Gentner, 1983), which describes analogy formation as the process of establishing structural alignment between entities and expressions in a source and target domain. Structure-Mapping Theory imposes four main constraints on this process:

1. **Structural Consistency**: The mapping must be one-to-one. E.g., if `Pressure`→`Temp` then we cannot also have `Diameter`→`Temp`.

2. **Parallel Connectivity**: If two expressions correspond, then all of their arguments correspond. E.g., a commitment to `Flow(beaker, vial, water, pipe)`→`Flow(coffee, icecube, heat, bar)` entails a commitment to beaker→coffee and water→heat, which would for instance forbid, `Liquid(water)`→ `Liquid(coffee)`.

3. **Tiered Identicality**: Matches between same identity expressions e.g., `Flow`→`Flow`, are prefferred over ones with different identities e.g., `Pressure`→`Temp`.

4. **Systematicity**: (explicitly expressed here in two parts), (a) the preferred (higher scoring) mappings are those that align larger systems of relations, (b) especially ones consisting of higher-order relations with nested children.

2.2 SME's Implementation

SME applies these four principles to its structure-mapping implementation (Forbus et al., 2017) in a three-phase process that relies heavily on a bias toward higher-order structure:

Match hypothesis generation: Enumerate all pairs of expressions in the source and target, storing pairs of match hypotheses that can plausibly match with one another. The rules for candidate plausibility are often configured on a domain-by-domain basis. Expressions must have the same arity (number of arguments), and constraints are set on which types of predicates can form correspondences. Each match hypothesis is assigned a weight reflecting its likelihood.

Kernel Formation: Each root match hypothesis (ones that are not arguments of others) seeds the formation of kernels which are built up by collecting the mapping entailed by their descendants. Each kernel is essentially a partial mapping, and each is assigned a Structural Evaluation Score (SES) (Forbus & Gentner, 1989) that aggregates the weights of its constituent match hypotheses in a manner that heavily favors kernels with deeper nested structures.

Kernel Merging: Beginning with the highest-scoring kernel, kernels are greedily assembled (Forbus & Oblinger, 1990) into complete matches by serially merging the highest-scoring kernel that is consistent with the match so far. Restarting the process from other seeds (lower-scoring initial kernels) yields a handful of alternative interpretations (three by default).

As we discuss later, SMTB aligns with structure-mapping Theory and SME, along these four constraints, but is considerably more relaxed than SME on (4b). SMTB does not explicitly privilege mappings with deeper hierarchies, although it tends to find them anyway because deeper hierarchies tend to create larger systems of overlap.

3. When Privileging "Higher-Order" Correspondences can be a Failure Mode

In our experience, we have not found that an explicit reliance on higher-order structure, such as SME kernels, is necessary for effective structure-mapping. Many routine situations require some form of associative partial alignment between representations, but rigidly privileging higher-order structures is either impossible, arbitrarily biased, or entirely incorrect. For instance:

When there is no higher-order structure to exploit, there is no way to bias structural mappings toward higher-order correspondences. This is not an uncommon situation. A broad survey of cognitive systems would likely find that flat first-order logic-like representations are more common than nested ones—not because flatness is preferable, but because hierarchy isn't always necessary. Classical planning, for instance, generally uses first-order representation languages (e.g., STRIPS/PDDL). Hierarchy is not a prerequisite for relational structure; as long as first-order expressions share entities, they have relational structure, and structure-mapping ought to work in the absence of higher-order structure. However, in these cases SME kernels reduce to individual match hypotheses, removing any differentiating signal for kernel seeding and merging, and SME becomes equivalent to greedy merging over candidate expression pairs; a strategy unlikely to consistently find good mappings in larger problems.

***Higher-orderness* is often an arbitrary side effect of notation**. In predicate logic notation, higher-order expressions encapsulate lower-order expressions. For instance, a second-order expression such as `Greater(Temp(coffee),Temp(icecube))` has first-order expressions `Temp(coffee)` and `Temp(icecube)` as arguments. However, we might reasonably question whether characterizing things as *higher* or *lower* solely on the basis of their referents ought to be given special treatment in determining structural connectedness. Predicate logic imparts *higher-orderness* on objects as a side effect of its notation; this is, however, far from an exhaustive treatment of hierarchy. Predicate logic notation does not, for instance, treat predicates as *higher* on the basis of derivational provenance, concept hierarchies, or part-whole relationships. `Father(fred, pops)` is equally first-order to `Parent(fred, pops)` and `IsMan(pops)`, despite the first being derivable from the others (from prior knowledge). Structure-mapping systems do not typically exploit these kinds of hierarchies, which are not captured by predicate logic's notation. And, perhaps they shouldn't. Our position is that relational connectedness—just the fact that objects are connected by some relation, whether nested or not—ought to be sufficient for establishing structural connectedness. SMTB, for instance, has no special reliance on higher-orderness, as SME does, allowing it to be fairly agnostic to data representation. SMTB will happily find good mappings over frame-based object-oriented ontologies, first-order predicates, and higher-order nested predicates all the same.

Relational cycles complicate which things are *higher*. In a pure hierarchy (e.g., a tree of objects), directed relationships clearly mark objects as *higher* or *lower* than one another. Add a relation pointing in the opposite direction, and you have a cycle, and the clarity of which things are

higher is lost. Predicate logics often forbid this as a notational side effect, simply because they lack unique identifiers to specify higher things so they can be passed as arguments to lower ones. This is a representation limitation; it shouldn't be a limitation of structure-mapping.

Higher correspondences should not require perfect child alignment. SME kernels are consistent mappings between perfectly alignable expression trees. We have found this policy to be overly restrictive. In an ongoing project, we use SMTB-based structure-mapping to simulate students' ability to learn from step-by-step math solutions by self-explaining the underlying changes between lines (Lee et al., 2026). Our simulated learners use structure-mapping to find structural similarities between subsequent lines to abduce the transformation that occurred between them. Structure-mapping aligns the changed parts around the unchanged ones, simplifying the search for a consistent generalization that explains the observed transformation. For instance, consider the following pair of equation lines, parsed into abstract syntax trees expressed in predicate logic notation:

Base line	**Target line**
$4x + 1 = 3^2$	$4x + 1 = 9$

`Equal(`
`  Add(`
`  Mul(Num(4), Sym(x)),`
`  Num(1)),`
`  Pow(Num(3), Num(2)))`
	`Equal(`
	`  Add(`
	`  Mul(Num(4), Sym(x)),`
	`  Num(1)),`
	`  Num(9))`

Table 2: Color coded structure map of $4x + 1 = 3^2$ to $4x + 1 = 9$ expressed in predicate logic.

Both SME and SMTB succeed at mapping 4x+1 on the left-hand side of the two equations (indicated by `Add`→`Add`). However, SME discards all other correspondences and thus fails to recognize that the only difference is localized to the right-hand side. An ideal mapping in this case should recognize that `Pow`→`Num(9)` (this kind of task requires loose match hypotheses to permit possibilities like these). And that particular hypothesis should gain support by its shared placement on the right-hand side of the shared higher-order `Equal`→`Equal`. However, since SME requires kernels to have valid argument correspondences all the way down, it cannot generate a kernel for `Equal` and thus misses the shared structure. The fact that the first-order expressions `Num(3)` and `Num(2)` have no counterparts in the target makes it impossible for SME to build a higher-order kernel. SMTB treats the higher-order structure more loosely, and thus avoids this failure mode.

4. SMTB: structure-mapping with Tight Bounds

The primary difference between SMTB and SME is that SMTB operates incrementally over a smaller unit of correspondence, assigning entities in the base to entities in the target at each search step, rather than assigning expressions to expressions or assigning sets of expressions collectively with kernels. A one-to-one entity assignment selects cells such that no assignment shares the same row or column. Table 3 shows the heat-flow analogy example, restated in terms of integer identifiers for each entity, an *identity mapping* (e.g. $0 \to 0, 1 \to 1, \ldots$ etc.) over the first 7 entities in the

Base: water flow		**Target: heat flow**	
0: beaker	8: Diameter(0)	0: coffee	8: Liquid(0)
1: vial	9: Diameter(1)	1: icecube	
2: water	10: Greater_2(8, 9)	2: heat	
3: pipe	11: Cause(6, 7)	3: bar	
4: Pressure(0)	12: Liquid(2)	4: Temp(0)	
5: Pressure(1)	13: Flat-Top(2)	5: Temp(1)	
6: Greater_1(4, 5)		6: Greater(4, 5)	
7: Flow(0, 1, 2, 3)		7: Flow(0, 1, 2, 3)	

Table 3: The entities of Table 1, numbered.

base and target is the ideal mapping. Note that in this context we mean "entities" to include both expression predicates (e.g. Greater_1(4, 5)) and base entities (e.g. beaker).

Just like in SME, SMTB begins with a set of match hypotheses (e.g., 6: Greater_1(4, 5)→ 6: Greater(4, 5) or 10: Greater_2(8, 9)→ 6: Greater(4, 5)) between expressions. SMTB accepts different configuration options args_only, group, or pairwise controlling how these hypotheses are interpreted as one or more ExpressionPair objects, with a weight ϕ_k. For instance, in group mode (the default), a match hypothesis 10: Greater_2(8, 9)→ 6: Greater(4, 5)) is interpreted as an ExpressionPair $[10, 8, 9] \rightarrow [6, 4, 5]$ which must be supported by $10 \rightarrow 6$, $8 \rightarrow 4$, and $9 \rightarrow 5$. In arg_only the parent is excluded, and the resulting ExpressionPair is $[8, 9] \rightarrow [4, 5]$. In pairwise multiple ExpressionPair objects are created connecting the parent to each argument individually; in this case $[10, 8] \rightarrow [6, 4]$ and $[10, 9] \rightarrow [6, 5]$. Optionally, SMTB can be provided with independent candidate EntityPairs with weights $e_{i,j}$. SMTB's objective is to find a mapping $M(i) \rightarrow j$ that maximizes the $e_{i,j}$ and ϕ_k contributions of mutually consistent ExpressionPairs:

$$S(M) = \sum_i e_{i,M(i)} + \sum_{T(k,i,M(i))} C(M,k)\phi_k \tag{1}$$

where $T(k, i, j)$ is an indicator function for when ExpressionPair k has entity support for $i \rightarrow j$, and $C(M, k)$ is an indicator function accepting ExpressionPairs consistent with M.

The main difficulty with incremental entity assignment is determining which assignments will successfully support the highest sum of ExpressionPair weights ϕ_k in an ideal match. An ExpressionPair is fully supported only when all of its supporting assignments have been made. Thus, it is difficult to know exactly which assignments will contribute to successfully locking in each ϕ_k, and the scope of realizable ExpressionPairs is often unclear until several assignments later. Some heuristic $h(i, j)$ is needed to guide the choice of assignments $i \rightarrow j$ toward a good mapping. Ideally, this heuristic is *admissible*; i.e., it is an upper bound on the score contribution of each assignment (Pearl, 1984) in a completed mapping.

4.1 Conceptual Framing with a Naive Upper Bound

A naively admissible $h(i, j)$ is to assume that all candidate ExpressionPairs are realizable by distributing the weight of each ϕ_k across its n_k supporting entity assignments.

		Target: heat flow								
		0	1	2	3	4	5	6	7	⋯
Base: water flow	0: `beaker`	**1.67**	1.00	1.00	1.00	–	–	–	–	⋯
	1: `vial`	1.00	**1.67**	1.00	1.00	–	–	–	–	⋯
	2: `water`	2.00	1.00	**1.67**	1.00	–	–	–	–	⋯
	3: `pipe`	1.00	1.00	1.00	**1.67**	–	–	–	–	⋯
	4: `Pressure(0)`	–	–	–	–	**1.67**	1.00	–	–	⋯
	5: `Pressure(1)`	–	–	–	–	1.00	**1.67**	–	–	⋯
	6: $\texttt{Greater}_1\texttt{(4, 5)}$	–	–	–	–	–	–	**1.67**	–	⋯
	7: `Flow(0, 1, 2, 3)`	–	–	–	–	–	–	–	**1.67**	⋯
	8: `Diameter(0)`	–	–	–	–	1.67	1.00	–	–	⋯
	9: `Diameter(1)`	–	–	–	–	1.00	1.67	–	–	⋯
	10: $\texttt{Greater}_2\texttt{(8, 9)}$	–	–	–	–	–	–	1.00	–	⋯
	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋱

Table 4: $h_{naive}(i, j)$ for heat flow example. Ideal mapping bolded along diagonal. The `water`→ `coffee` distractor (score 2.00) is best in its column, from `Liquid(water)`→`Liquid(coffee)`.

$$h_{\text{naive}}(i, j) = e_{i,j} + \sum_{T(k,i,j)} C(M, k)\phi_k/n_k \tag{2}$$

This is a very loose upper bound on the expected weight contribution of each assignment, because, in practice, many candidate `ExpressionPairs` are mutually exclusive. For instance, if we accept $[10, 8, 9] \to [6, 4, 5]$ then we cannot also accept $[6, 4, 5] \to [6, 4, 5]$, because the mapping must be one-to-one. Table 4 shows this naive heuristic applied over a portion of the assignment matrix. The heatflow example is configured with 1.0 weight for each entity candidate pair, and $\phi_k = \frac{2}{3}n_k$ for $n_k > 1$ and $\phi_k = 0.5$ when $n_k = 1$.[2]

Table 4 shows the naive heuristic applied at the first step of incremental backtracking search; thereafter, the heuristic matrix is recomputed after each assignment. With greedy backtracking search, the first assignment would choose $2 \to 0$ with $h(2, 0) = 2.0$, a locally optimal but globally non-optimal choice. And after recomputing $h(i, j)$, it would zero out column 0 and row 2, since all $0 \to j$ and $i \to 2$ would be inconsistent. As the search continues, several complete non-optimal mappings are explored, eventually backtracking to the start, where $0 \to 0$ would eventually lead to an optimal mapping. However, full backtracking search would continue searching after that until the entire space is exhaustively searched.

The heuristic value of any cell can change after any assignment, since some `ExpressionPairs` can be invalidated, which affects other rows and columns. This fact prevents us from viewing structure-mapping as equivalent to a Linear Assignment Problem (LAP)—a class of optimization problems with fast polynomial-time solutions, such as the Hungarian Algorithm (Kuhn, 1955), where an optimal one-to-one mapping is found over a cost matrix of paired candidate mappings. LAP solvers will find consistent but naively greedy structure maps. Structure-mapping is closer to a Quadratic Assignment Problem (QAP) (Koopmans & Beckmann, 1957), which is known to be

2. These particular weight choices aren't particularly important; they simply slightly favor non-unitary predicates when the weights are distributed, and larger predicates' weight isn't diluted upon distribution.

NP-Hard, and is often applied to optimal graph alignment problems (Conte et al., 2004). However, structure-mapping is arguably even harder than QAP since it uses arbitrary n-ary relations (instead of only binary ones that can be treated as graph edges), and higher-order relations—i.e., expressions can also be entities, so there is not always a clean node vs. edge distinction.

This naive approach using h_{naive} is wasteful on several accounts: 1) after each assignment it recomputes the heuristic by repeated summation instead of incremental updates, 2) its values reflect local relational connectivity instead of connectivity as part of larger connected systems, 3) it uses a bound which is far too loose because it sums the weights of `ExpressionPairs` which can never be realized simulatenously in the same one-to-one mapping, and 4) we have motivated it within an exhaustive backtracking search which would have $O(n!)$ runtime relative to problem size. The main features of SMTB fix issues 1 and 3 by tightening the heuristic and updating it incrementally. Issue 2 can be addressed separately by adding an extra term to the heuristic (described in Section 4.3) that weights total connectivity similarly to SME's Structure Evaluation Score (SES). With these features applied, issue 4 no longer becomes a problem because the heuristic is strong enough to find good matches greedily without backtracking.

4.2 Tightening the Bounds by Peeking Ahead

SMTB provides a much tighter upper bound by explicitly accounting for the fact that each entity assignment $i \to j$ effectively forces some future assignment $u \to v$ to complete a subset of the `ExpressionPairs` that it began to fill in. We can formulate a tighter $h(i,j)_{tight}$ bound by pre-computing the maximal weight contribution of realizable ϕ_k supported by each $i \to j$. If we were to determine this value precisely, we would need to find the optimal mapping after pinning each $i \to j$, which is, of course, as hard as the original problem. Instead, we can make a first-order approximation that looks one step ahead of the current partial mapping M. Consider the set of `ExpressionPairs` in Table 5 as an example.

`[0,1,2]`→`[5,6,8]` $\phi_0/n_k = 1/3$	`[0,1,4]`→`[10,7,9]` $\phi_4/n_k = 2/3$
`[0,1,2]`→`[5,7,9]` $\phi_1/n_k = 1/3$	`[0,2,4]`→`[10,8,9]` $\phi_5/n_k = 1/3$
`[0,3,4]`→`[5,6,8]` $\phi_2/n_k = 1/3$	
`[0,3,4]`→`[5,7,9]` $\phi_3/n_k = 1/3$	
Candidate: $0 \to 5$	**Candidate:** $0 \to 10$
$h_{\text{naive}}(0,5) = 4/3$	$h_{\text{naive}}(0,10) = 3/3$

Table 5: Example `ExpressionPairs` for two candidate assignments $0 \to 5$ and $0 \to 10$ with their corresponding naive heuristic values 4/3 and 3/3.

The `ExpressionPairs` on the left and right are supported by the candidate assignments $0 \to 5$ and $0 \to 10$. Ignoring any $e_{i,j}$ contributions, the naive heuristic would assign $h(0,5) = 4/3$ and $h(0,10) = 3/3$, meaning it would choose $0 \to 5$ as the more favorable next match. However, no one-to-one mapping can simultaneously assign `[0,1,2]`→`[5,6,8]` and `[0,1,2]`→`[5,7,9]`, or `[0,3,4]`→`[5,6,8]` and `[0,3,4]`→`[5,7,9]`, so in reality a complete mapping commited to $0 \to 5$ would only support half of the weight $2/3$ suggested by the naive heuristic. It would have been better to select $0 \to 10$, which accumulates a weight of $3/3$ without contradiction. SMTB

calculates a tighter heuristic by maintaining a dependency matrix for each $i \to j$ to estimate the maximum `ExpressionPair` weight that can be locked in while keeping a candidate assignment's dependent candidate assignments $u \to v$ consistently one-to-one.

Candidate: $0 \to 5$

	6	**8**	**7**	**9**	$\mathbf{r_u}$
1	$1/6$	–	$1/6$	–	$1/6$
2	–	$1/6$	–	$1/6$	$1/6$
3	$1/6$	–	$1/6$	–	$1/6$
4	–	$1/6$	–	$1/6$	$1/6$
$\mathbf{c_v}$	$1/6$	$1/6$	$1/6$	$1/6$	$\mathbf{2/3}$

Candidate: $0 \to 10$

	7	**9**	**8**	$\mathbf{r_u}$
1	$1/3$	–	–	$1/3$
4	–	$1/2$	–	$1/2$
2	–	–	$1/6$	$1/6$
$\mathbf{c_v}$	$1/3$	$1/2$	$1/6$	$\mathbf{3/3}$

Table 6: Dependency matrices D for two candidate assignments.

Each cell in a dependency matrix D distributes `ExpressionPair` weight once more by another factor of $1/(n_k - 1)$, since there are $(n_k - 1)$ dependency slots for each `ExpressionPair`, and computes the row maximums $r_u = \max_v D_{u,v}$ and column maximums $c_v = \max_u D_{u,v}$. The final heuristic value is the minimum of the sums of those maximums:

$$h(i,j)_{tight} = \min(\sum_u r_u, \sum_v c_v) \tag{3}$$

Table 6 shows the dependency matrices for $0 \to 5$ and $0 \to 10$ and the tighter heuristic values $2/3$ and $3/3$, which in this case are exactly the maximum distributed weights achievable by each candidate assignment. The tight SMTB heuristic correctly prefers $0 \to 10$ over $0 \to 5$.

4.3 Preferring Global Structural Connectivity with Paired Walks

The tight SMTB heuristic helps privilege assignments that maximize realizable structural connectivity locally, but carries no signal about the size of the total connected systems surrounding them. SME's Structural Evaluation Score (SES) evaluates connected system size through kernels realized via higher-order predicates. In first-order predicate encodings, like family trees encoded only with `Parent(child_id, parent_id)`, SES would provide no helpful signal. It couldn't help determine, for instance, if some family tree A was more structurally similar to trees B or C.

SMTB takes a simpler approach which uses paired walks to measure the shared connectivity around each $i \to j$ assignment. Setting `local_walk_depth` controls how many recursive steps L are used to calculate a bias term $b_{i,j}$ reflecting the shared connectivity in neighboring entities. The base case $L = 0$ adds $e_{i,j}$ to $b_{i,j}$, which includes some weight associated with entities sharing the same type (by default 1.0; less if they only share a parent type) plus a small amount of weight for any constant attribute values they have in common (.3 by default). At L=1 and greater, the walk progresses outward. Entity-valued attributes are checked for candidate entity map compatibility (i.e., a $e_{i,j}$ weight must be defined between them). Then their base weight $e_{i,j}$ is added to $b_{i,j}$, and if the depth limit has not been reached, they are added to the frontier so the walk can recurse and collect the pair weights of their neighbors, and so on. Walks grow geometrically with respect to

their walk depth L; however, a covered set is maintained so no entity in the base or target is added to the frontier twice, bounding walk length to at most the domain size.

Unlike SES, these paired walks can reflect shared connectivity even among first-order predicates. By default, they follow parent-argument relationships bidirectionally, so if entity/expression `Parent(fred, pops)` in the base domain takes entity `fred` as an argument, then a walk centered around `fred` can traverse backward through that connection and add `Parent(fred, pops)` to its frontier, assuming, of course, that the walk has paired them with entities in the target sharing an analogous parent-argument relationship. Unlike SES, this approach does not depend upon higher-order predicates to connect lower-order ones.

To align $b_{i,j}$ values with the smaller scale of $h(i,j)$, the bias matrix is normalized by it's maximum $\hat{b}_{i,j} = b_{i,j} / \max_{i,j}(b_{i,j})$, and given a fractional contribution α_d in the final heursitic, which decays exponentionally as a function of the current search depth d (i.e. how many assignments have already been made).

$$\alpha_d = \alpha_0 e^{-d/\tau} \tag{4}$$

$$h(i,j)_{L=n} = \frac{\alpha_d \hat{b}_{i,j} + h(i,j)_{tight}}{\alpha_d + 1} \tag{5}$$

The idea behind decaying the $\hat{b}_{i,j}$ contribution is that each $b_{i,j}$'s measure of system-level connectivity is more useful early on for selecting the initial assignments that seed larger connected groups. However, $b_{i,j}$ values are only calculated at start-up, so they become less helpful once some assignments have rendered portions of the candidate connections they accounted for unrealizable. $b_{i,j}$ also tend to be similar among connected entities, so they are more helpful for seeding group alignment than for maximally finishing aligning the constituent entities in a pair of connected systems.

5. Methods

We evaluate SMTB on the SME corpus, a set of domains used in prior SME research (Forbus et al., 2017), relative to SME v4 implemented in Allegro Common Lisp. We run both models head-to-head on the same device in a single thread, and compare the quality of the mappings they produce. The SME corpus consists of 5,845 base-target domain pairs with 5 sub-corpora `ap-physics` (1,137) (Klenk & Forbus, 2009), `geometry` (873), `moraldm` (420) (Dehghani et al., 2008), `oddity` (3,411) (Lovett & Forbus, 2017), and `thermo` (4) (Forbus, 2001). Since SMTB and SME optimize over different objectives, we compare how each resulting mapping scores on both scales. The **SMTB objective** is $S(M)$ (equation 1), the sum of all $e_{i,j}$ and ϕ_k contributions satisfied by each mapping. The **SME objective** is the sum of Structural Evaluation Scores (SES) of the realized match hypotheses. We normalize these scores by the value SME v4 achieves on each scale.

Some mappings that SMTB can produce would be forbidden by SME. These include situations like the algebraic equations in Table 2, where higher-order predicates are mapped between the base and target that would have failed SME's kernel construction phase due to the absence of mappings in lower-order constituents. These *kernel violations* contradict an assumption of SME's approach, but not the main assumptions of structure-mapping theory like *Structural Consistency* (i.e., one-to-one mappings) and *Parallel Connectivity*. We record the percentage of entity correspondences in

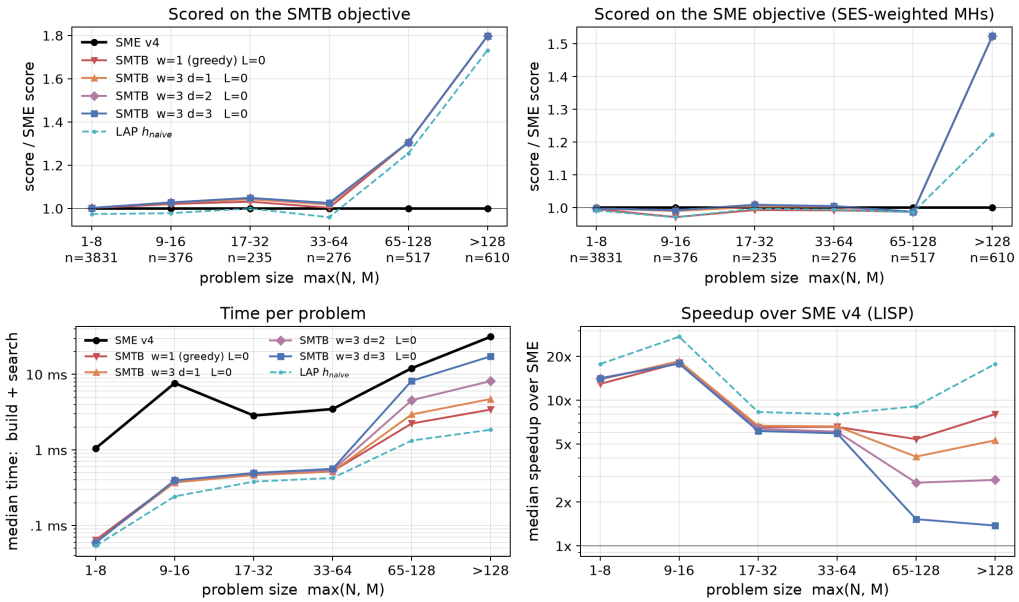


Figure 1: Mapping quality ratio (top) under SMTB and SME's objectives relative to SME, and model solve time (bottom), including match hypothesis construction and search time. All results are graphed as a function of problem size (max number of entities between base and target).

SMTB's mappings that SME would consider **kernel violations**, and also report score results with those entity assignments removed from the final mapping.

We configure SMTB to interpret corpus problems precisely as SME does. We translated SME corpus problems into JSON (so CRE's Python extension can read them) and set up SMTB to correctly interpret a wide variety of configuration options hard-coded into each problem description. Admittedly, this was tedious work, and language models helped with much of it. However, we verified that the starting match hypotheses generated by SMTB and SME were identical in this setup. Our timing measurements discount these translation steps (which are mostly written in Python). However, we measure the *build* time associated with forming match hypotheses from the base and target domains after they have been initialized in SMTB's native format, as well as the *search* time for each mapping. The SME v4 runtimes are analogous and rely on its built-in internal timers.

SMTB is run with a bounded backtracking search parameterized by a branch width w (how many candidate columns a row may try) and a branch depth d (how deep the search may backtrack); $w{=}1, d{=}1$ is therefore purely greedy assignment with no backtracking at all. We also report a `LAP` baseline that solves a single linear assignment problem over the naive heuristic h_{naive}.

6. Results

The top two graphs of Figure 1 show SMTB's mapping score ratio relative to SME by problem size. For small problems (<64 entities), SMTB and SME perform nearly the same, with variants of SMTB showing at most $1.05\times$ better scores than SME evaluated on SMTB's objective and $0.99\times$ and

		% Score gain vs. SME (SMTB objective)				**% Score gain vs. SME** (SME objective + Violations Removed)				**% Kernel Violation**			
Domain	n	L=0	L=1	L=3	L=5	L=0	L=1	L=3	L=5	L=0	L=1	L=3	L=5
`ap-physics`	1137	46.4	48.5	48.5	48.0	34.0	40.2	41.9	41.8	0.50	0.66	0.61	0.61
`geometry`	873	0.8	1.0	0.8	0.9	−2.8	−1.1	−0.9	−1.8	5.41	6.27	5.33	5.67
`moraldm`	420	3.0	3.5	3.4	3.3	1.2	1.5	1.4	1.6	0.14	0.14	0.14	0.00
`oddity`	3411	0.5	0.3	0.5	0.6	−1.6	−1.3	−1.1	−1.3	3.61	5.84	4.17	4.39
`thermo`	4	5.3	6.6	5.9	4.4	−6.0	−1.3	0.0	0.0	9.16	10.58	4.74	5.84
ALL	5845	**21.6**	**22.5**	**22.6**	**22.4**	**10.3**	**14.7**	**16.0**	**15.9**	**1.24**	**1.78**	**1.38**	**1.44**

Table 7: Percent gain in mapping score over SME for SMTB w=3, d=1, varied by local walk depth $L = \{0, 1, 3, 5\}$. Gains over SME are reported under SMTB's objective and under SME's objective. This time (unlike Figure 1), for the SME objective, entity correspondences that SME kernels would have forbidden are removed. We report the percentage of these *kernel violation* correspondences, which are small overall 1.24-1.78%, albeit as much as 10.58% for some domains and configurations.

1.01× SME's score evaluated on the SME objective. For medium problems with 65–128 entities, SMTB reaches 1.31× on the SMTB objective, and for large problems with more than 128 entities it reaches 1.80×. For these large problems, SMTB also beats SME by 1.52× on the SME objective. The LAP solver over h_{naive} is consistently slightly worse than SMTB, but by all means a viable solution that also beats SME.

The bottom two graphs of Figure 1 report total solution time over the whole SME corpus. SME takes 70.95 s; purely greedy SMTB takes 5.97 s and backtracking search with w=3, d=3 (for 27 total search arms) takes 30.99 s. w=3, d=1 is a good tradeoff with a consistent 5-15x speedup over SME across problems sizes. This is also the configuration most similar to SME in terms of search branching, since, like SME, it chooses 3 different initial seed assignments.

Adding paired walks (i.e., $L \geq 1$, see Table 7) marginally improves score quality, with a small cost to overall solve time. SMTB with w=3, d=1 scores 21.6% above SME at $L = 0$ and 22.6% above SME at $L = 3$, with a total corpus runtime: 8.47s for $L = 0$, 8.63s for $L = 3$, and 9.11s for $L = 10$. Paired walks, in some cases, help SMTB yield mappings better aligned to SME's approach. For instance, Table 7 also reports *kernel violation* rates (right columns). When reporting SME's objective, we remove kernel violation mappings before scoring (middle columns). $L = 5$ cuts those kernel violations from 9.16% to 5.84% in `thermo`, and from 0.14% to 0.0% in `moraldm`, and appreciably improves SME scores in `ap-physics`, from 34.0% above SME at $L = 0$ to 41.9% above SME at $L = 3$, albeit with a slight increase in kernel violations.

Table 7 also shows that most of SMTB's overall advantage under the SME objective comes from domains where SME hasn't already saturated the available entities. SMTB has the largest gains in the `ap-physics` domain, with smaller benefits in `moraldm` and small losses or equivalence elsewhere in the more saturated domains. Table 8 shows that the gains in `ap-physics` and `moraldm` are from SMTB finding mappings that include larger consistent paired systems of base entities

	Depth	Avg. #		Base Mapped		Exprs. Mapped	
Domain		Base	Expr.	SMTB	SME	SMTB	SME
`ap-physics`	7.1	47.7	100.2	**20.1**	13.5	**23.0**	13.3
`geometry`	1.4	2.3	1.7	2.1	2.1	1.3	1.3
`moraldm`	4.0	16.1	17.2	**15.0**	9.3	1.9	1.8
`oddity`	1.5	2.4	1.8	2.4	2.4	1.7	1.7
`thermo`	7.5	10.8	134.8	9.0	7.8	69.8	61.8
ALL	**2.7**	**12.2**	**22.2**	**6.7**	**5.0**	**5.9**	**3.9**

Table 8: Structure of each corpus domain, and how many base entities (e.g. `coffee`) and expressions (e.g. `Liquid(coffee)`) are mapped by SMTB and SME. In domains with more nested higher-order structure like `ap-physics` and `moraldm`, SMTB with $w{=}3, d{=}1, L{=}0$ tends to find mappings that include about 50% more mutually consistent correspondences.

and expressions. SMTB's objective is directly proportional to the number of correspondences, but SME's SES-based objective heavily favors systems with more higher-order structure (as opposed to just big ones). The fact that SMTB beats SME under the SME objective in these larger nested domains verifies that SMTB is not scoring higher as a result of pursuing a different objective, or from allowing more relaxed mappings (like our algebra example in Table 2). Kernel violation removal cuts out all relaxed correspondences like these. This leaves us with the conclusion that SMTB is indeed better than SME at maximizing the SME objective, and in general, at forming good mappings in large domains with deep hierarchical structure.

7. Discussion

We have promoted SMTB largely on practical grounds: it is implemented in a fast, Python-accessible toolkit, it works flexibly across different representational choices, and it finds good mappings in large problems (>128 entities) with higher-order structure. With respect to theory, SMTB diverges from structure-mapping Theory and SME on a few small details: 1) whether or not higher-orderness should be privileged explicitly (point 4b in Section 2.1); and 2) whether mapping correspondences are formed serially, entity-by-entity, or by groups of higher-order connected systems (i.e. kernels). For 1) SMTB offers evidence that good higher-order mappings can be found without giving them special treatment. Perhaps then, a theory of structural alignment based only on connectedness is sufficient; it certainly admits more flexible correspondences, which we've found necessary in some cases (e.g. the algebra example). For 2) SMTB assigns correspondences over smaller mental units. Future human experiments—if they could deconfound a higher-orderness preference from a preference for larger connected systems—may be able to clarify where humans fall along these points.

8. Conclusion

We are, however, more excited by the practical trajectory of CRE and SMTB. Cognitive systems offer powerful untapped affordances for artificial intelligence, since they tend to solve precisely

the problems that riddle naive data-driven neural network-based approaches like LLMs. Symbolic cognitive systems often learn considerably faster than neural approaches like reinforcement learning and typically induce interpretable, and robust knowledge structures. We should be reminded that the meteoric improvements in AI capabilities over just the last 5 years are largely the product of massively scaling a narrow AI paradigm. The research preceding that scaling was accelerated by good tools (e.g., PyTorch & Tensorflow). It is not unreasonable to believe that cognitive systems could experience a similar trajectory, if the right tools set the stage for them to scale in the same way. Structure-mapping would play a key role in such a revolution since it is a key component of powering rapid human-like learning and flexible pattern matching capabilities with over symbolic knowledge representations.

References

Anderson, J. R., Matessa, M., & Lebiere, C. (1997). ACT-R: A theory of higher level cognition and its relation to visual attention. *Human–Computer Interaction*, *12*, 439–462.

Cen, H., Koedinger, K., & Junker, B. (2006). Learning factors analysis–a general method for cognitive model evaluation and improvement. *International Conference on Intelligent Tutoring Systems* (pp. 164–175). Springer.

Conte, D., Foggia, P., Sansone, C., & Vento, M. (2004). Thirty years of graph matching in pattern recognition. *International Journal of Pattern Recognition and Artificial Intelligence*, *18*, 265–298.

Dehghani, M., Tomai, E., Forbus, K. D., & Klenk, M. (2008). An integrated reasoning approach to moral decision-making. *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence* (pp. 1280–1286).

Falkenhainer, B., Forbus, K. D., & Gentner, D. (1989). The structure-mapping engine: Algorithm and examples. *Artificial intelligence*, *41*, 1–63.

Forbus, K. D. (2001). Exploring analogy in the large. In D. Gentner, K. J. Holyoak, & B. N. Kokinov (Eds.), *The analogical mind: Perspectives from cognitive science*. Cambridge, MA: MIT Press.

Forbus, K. D., Ferguson, R. W., Lovett, A., & Gentner, D. (2017). Extending sme to handle large-scale cognitive modeling. *Cognitive Science*, *41*, 1152–1201.

Forbus, K. D., & Gentner, D. (1989). Structural evaluation of analogies: What counts? *Proceedings of the Eleventh Annual Conference of the Cognitive Science Society* (pp. 341–348).

Forbus, K. D., & Hinrichs, T. (2017). Analogy and relational representations in the companion cognitive architecture. *AI Magazine*, *38*, 34–42.

Forbus, K. D., & Oblinger, D. (1990). Making sme greedy and pragmatic. *Proceedings of the Twelfth Annual Conference of the Cognitive Science Society* (pp. 61–68).

Forbus, K. D., Usher, J., Lovett, A., Lockwood, K., & Wetzel, J. (2011). Cogsketch: Sketch understanding for cognitive science research and for education. *Topics in Cognitive Science*, *3*, 648–666.

Gentner, D. (1983). Structure-mapping: A theoretical framework for analogy. *Cognitive Science*, *7*, 155–170.

Gentner, D. (2010). Bootstrapping the mind: Analogical processes and symbol systems. *Cognitive Science*, *34*, 752–775.

Gentner, D., & Forbus, K. D. (2011). Computational models of analogy. *Wiley Interdisciplinary Reviews: Cognitive Science*, *2*, 266–276.

Gentner, D., & Forbus, K. D. (2025). The structure-mapping engine: A multidecade interaction between psychology and artificial intelligence. *Current Directions in Psychological Science*, *35*, 148–155.

Käser, T., & Alexandron, G. (2024). Simulated learners in educational technology: A systematic literature review and a Turing-like test. *International Journal of Artificial Intelligence in Education*, *34*, 545–585.

Klenk, M., & Forbus, K. (2009). Analogical model formulation for transfer learning in ap physics. *Artificial intelligence*, *173*, 1615–1638.

Koopmans, T. C., & Beckmann, M. (1957). Assignment problems and the location of economic activities. *Econometrica*, *25*, 53–76.

Kuehne, S. E., Forbus, K. D., Gentner, D., & Quinn, B. (2000). Seql: Category learning as progressive abstraction using structure mapping. *Proceedings of the Twenty-Second Annual Conference of the Cognitive Science Society* (pp. 770–775).

Kuhn, H. W. (1955). The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, *2*, 83–97.

Laird, J. E. (2019). *The soar cognitive architecture*. MIT press.

Lee, S., MacLellan, C. J., & Weitekamp, D. (2026). Structure-mapping-guided self-explanation for learning mathematical procedures. *Proceedings of the Thirteenth Annual Conference on Advances in Cognitive Systems*.

Lovett, A., & Forbus, K. (2017). Modeling visual problem solving as analogical reasoning. *Psychological Review*, *124*, 60–90.

Maclellan, C. J., Harpstead, E., Patel, R., & Koedinger, K. R. (2016). The Apprentice Learner Architecture: Closing the loop between learning theory and educational data. *International Educational Data Mining Society*.

McLure, M. D., Friedman, S. E., & Forbus, K. D. (2015). Extending analogical generalization with near-misses. *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*.

Pearl, J. (1984). *Heuristics: Intelligent search strategies for computer problem solving*. Reading, MA: Addison-Wesley.

Weitekamp, D. (2025). CORGI: Efficient pattern matching with quadratic guarantees. *Proceedings of the Twelfth Annual Conference on Advances in Cognitive Systems* (pp. 212–225).

Weitekamp, D., Harpstead, E., & Koedinger, K. (2020). An interaction design for machine teaching to develop ai tutors. *CHI*.

Weitekamp, D., Harpstead, E., & Koedinger, K. (2024). AI2T: Building trustable ai tutors by interactively teaching a self-aware learning agent. *arXiv preprint arXiv:2411.17924*.

Weitekamp, D., & Koedinger, K. (2023). Computational models of learning: deepening care and carefulness in ai in education. *International conference on artificial intelligence in education* (pp. 13–25). Springer.

Weitekamp, D., MacLellan, C., Harpstead, E., & Koedinger, K. (2025a). Decomposed inductive procedure learning: Learning academic tasks with human-like data efficiency. *Proceedings of the Annual Meeting of the Cognitive Science Society*.

Weitekamp, D., Siddiqui, M. N., & MacLellan, C. J. (2025b). Tutorgym: A testbed for evaluating ai agents as tutors and students.